CSE 141L Final Submission

Kristopher Manalo, A17429503

Academic Integrity

Your work will not be graded unless the signatures of all members of the group are present beneath the honor code.

To uphold academic integrity, students shall:

- Complete and submit academic work that is their own and that is an honest and fair representation of their knowledge and abilities at the time of submission.
- Know and follow the standards of CSE 141L and UCSD.

Please sign (type) your name(s) below the following statement:

I pledge to be fair to my classmates and instructors by completing all of my academic work with integrity. This means that I will respect the standards set by the instructor and institution, be responsible for the consequences of my choices, honestly represent my knowledge and abilities, and be a community member that others can trust to do the right thing even when no one is watching. I will always put learning before grades, and integrity before performance. I pledge to excel with integrity.

_____

Kristopher Manalo
A17429503

0. Team

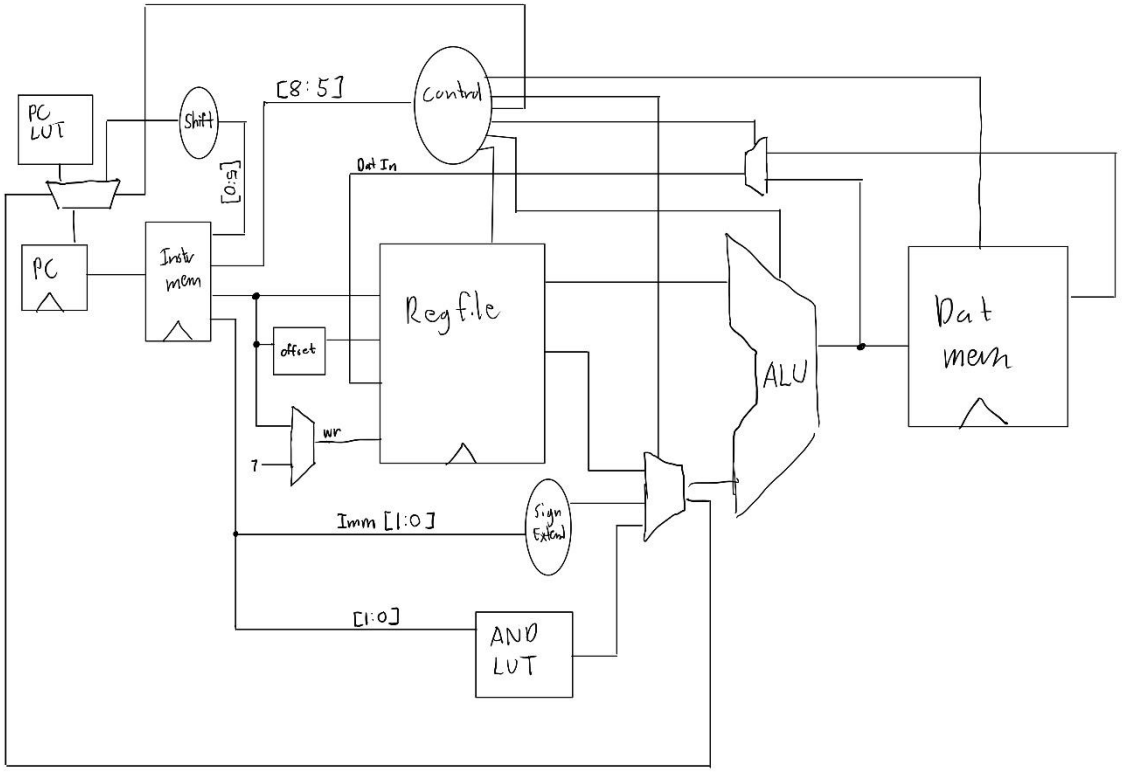
Team consists of one member

1. Introduction

Architecture name: Freaky Gojo

- The philosophy of design of this architecture is around the idea of locality. Operations between registers are likely to be close to each other hence register calculations are done between a source register and its offset register. Due to limited instruction width, the output is stored in an output register.
- Due the output limitation, my goal was to allow registers to quickly write to another register without having to access the machine's memory.
- The machine still has load/store capabilities making it a derivative of a load-store machine, with the philosophy to minimize loading and storing. With that considered the machine classification is load-lite machine.

2. Architectural Overview



The architecture will consist of a standard ALU, REG FILE, PC register, INSTR memory, and OPCODE decoder. In addition, several muxes will determine the register pointers for the register file. The architecture will also use LOOKUP tables to expand meaning of the small bit input for both branching and for AND Bitmasks.

3. Machine Specification

Instruction formats

S – Special behavior
R – Register
B – Branch
J – Jump
I3/I2 – Immediate
M – Memory

TYPE	FORMAT	CORRESPONDING INSTRUCTIONS
S	Special behavior	NOP, DON
R	2 bit opcode, 2-bit funct, 3-bit operand register, 2 bit operand register	AND, ADD, XOR
B	2 bit opcode, 1-bit funct, 3-bit operand register, 3-bit destination mapping	BEQ, BLT, BCO
J	3-bit opcode, 6-bit immediate	JMP
I#	3/4-bit opcode, 3-bit operand register, 3/2-bit immediate	ADI, LSL, LSR
M	3 bit opcode, 3 bit operand register, 3 bit operand register	LWD, SWD

Operations

NAME	TYPE	BIT BREAKDOWN	EXAMPLE	NOTES
AND = and with special bitwise maps	R	opcode (0000) 3-bit operand register (X) 2-bit register offset (X)	#Assume R6 has `0b1001_1101` #Assume R1 has `0b0000_1111` `AND r7, r1` ⇔ `0000_110_10` # After AND instruction, R7 now holds `0b0000_1101`	The last two bits define the register offset from the source register and stores it into register 7. 0b'00 RS + 1 0b'01 RS + 2 0b'10 RS + 3 0b'11 RS + 4
ADD = adds two registers	R	opcode (00) funct (01) 3-bit operand register (X) 2-bit register offset (X)	# Assume R3 has `0b0000_0001` # Assume R5 has `0b0000_1000` # Assume R7 has `0b1111_1111` `ADD r3, r5` ⇔ `00_01_011_01` # After ADD instruction, R7 now holds `0b0000_1001`	The last two bits define the register offset from the source register and stores it into register 7. 0b'00 RS + 1 0b'01 RS + 2 0b'10 RS + 3 0b'11 RS + 4 Adding by two duplicate registers will be interpreted into LSL 2 by the assembler
LWD = load word	M	opcode (01) funct (0) 3-bit operand register (X) 3-bit operand register (X)	# Assume R3 has `0b1111_1111` # Assume R4 has `0b0000_0000` # Assume dat_mem[256] has `0b0101_0101` `LWD r4, r3` ⇔ `01_0_100_011` # After LWD instruction, R4 now holds `0b0101_0101`	Loads dat_mem[RS] into RT: `RT <= dat_mem[RS]`

SWD = store word	M	opcode (01) funct (1) 3-bit operand register (X) 3-bit operand register (X)	# Assume R3 has 0b1111_1111 # Assume R4 has 0b1010_1010 # Assume dat_mem[256] has 0b0101_0101 SWD r4, r3 ⇔ 01_1_100_011 # After SWD instruction, dat_mem[256] now contains 0b1010_1010	Stores RT into dat_mem[RS]: RT => dat_mem[RS]
BEQ = branch if equal	B	opcode (10) funct (0) 3-bit operand register (X) 3-bit mapping (X)	# Assume R3 has 0b0000_1111 # Assume R7 has 0b0000_1111 # Assume PC = 0b00000_1000 # Assume “111” maps to -5. BEQ r3, #111 ⇔ 10_0_011_111 # After BEQ instruction, PC contains 0b0000_0011	The 3-bits will map to designated relative jump addresses to facilitate local branching of if, for, etc. The register provided will be compared to &register + 1. If equivalent, will go to offset or label.
BLT = branch less than	B	opcode (10) funct (1) 3-bit operand register (X) 3-bit mapping (X)	# Assume R3 has 0b0000_1110 # Assume R7 has 0b0000_1111 # Assume PC = 0b00000_1000 # Assume “111” maps to -5. BLT r3, #111 ⇔ 10_0_011_111 # After BLT instruction, PC contains 0b0000_0011	The 3-bits will map to designated relative jump addresses to facilitate local branching of if, for, etc. The register provided will be compared to &register + 1. If RS less than register + 1, will go to offset or label.

BCO = branch carry out	B	opcode (10) funct (10) 2-bit map to null (X) 3-bit mapping (X)	# Previous clock cycle # shift_carry_out = 0 BCO .Label # pc = 5 ⇔ 10_11_XX_011 .Label #pc = 10	The 3-bits will map to designated relative jump addresses to facilitate local branching of if, for, etc. The branch will be taken if previous clock cycle's carry_out from the ALU was flagged as 1.
JMP = jump	J	opcode (110) 6-bit jump buffer (X)	# Assume PC = 0b0000_1000 # Assume R1 has 0b1111_1111 JMP #28 ⇔ 1100_11100 # After JML instruction: # PC has 0b1111_0000 # R1 has 0b0000_1000	The instruction memory is divided into 32 sections where each jump buffer value is designated a section. This allows an escape from the restrictive control flow of the relative branches
ADI = add immediate	I3	opcode (111) 3-bit operand destination (X) 3-bit immediate (X)	# Assume R7 has 0b0000_1000 # Assume R3 has 0b0000_1000 ADI r3, #0 ⇔ 111_011_000 # After ADI instruction, R3 contains 0b0000_1000	Stores output into provided register. Adds immediate to R7. Available values of immediate are $[-4, 3]$. Allows for writing of registers without storing and loading from memory by adding by immediate of 0. Providing R0 as destination will cause R0 + Immediate => R7.

LSL = Rolling* shift left	I2	opcode (00) funct (10) 3-bit operand register (X) 2-bit Immediate (X)	# Assume R7 has `0b1100_0001` `LSL r7, #1` ⇔ `00_10_111_00` # After LSL instruction, R7 now holds `0b1000_0011`	The last two bits defines the amount of rolling done, the value is unsigned and cannot = 0. 0b'00 = 1 0b'01 = 2 0b'10 = 3 0b'11 = 4 Left shift by n bits are analogous to multiplication of 2^n, if the rolled bits are bit masked out.
XOR	R	opcode (00) funct (11) 3-bit operand register (X) 2-bit Immediate (X)	# Assume R6 has `0b1100_0001` and `R7 has 0b1111_1111` `LSR r6, r7` ⇔ `00_11_110_00` # After XOR instruction, R7 now holds `0b0011_1110`.	Bitwise XOR between rs and rt. Stores output into register 7.
NOP	S	UNDEFINED	`NOP` ⇔ 111111000	Marks next instruction to have no operation Is an assembler macro
DON	S	UNDEFINED	`DON` ⇔ 000111100	Marks program as complete

Internal Operands

Special registers:

- R0 – always evaluates to 0000_0000, cannot be overwritten
- R7 – destination register for R-type instructions
- [R1,R7] can all be overwritten through applicable instructions.

Control Flow (branches)

Absolute and relative jumps are supported. BEQ and BLT utilize relative addresses. JMP utilize absolute addresses. Maximum displacement from a branch instruction is 30 instructions. Larger jumps are implemented by utilizing JMP.

Labels are handled by the assembler, where the assembler must know to preprocess them by inclusion of the ‘-r’ flag.

Addressing Modes

Addressing into memory is indirect, where an address must first be stored into a register then used in a LWD or SWD instruction. To calculate an arbitrary address utilize LSL and ADI to get to desired value.

Example loading into Register 5 from dat_mem[67]:

```
LSL R0, R1 // Stores 0 into R7
ADI R7, 2
ADI R6, 2 // Inserts 4 into R6
LSL R6, R7 // Multiplies 4 by 2
LSL R6, R7 // Multiplies 8 by 2
LSL R6, R7 // Multiplies 16 by 2
LSL R6, R7 // Multiplies 32 by 2
ADI R6, 3 // Adds 3 to 64
LDW R5, R6
```

[

4. Programmer's Model

The machine, in some of its designs, operates in an unintuitive manner to how would expect MIPS to operate. Many 3 arguments instructions were shortened into 1.5 argument instructions and the invariant register of an argument may not be constant across instructions.

Vital to understanding this architecture is understanding the difference in the ADI instruction and its argument of destination register rather than (the standard across the architecture) source register. The ability to change the destination register is vital as the destination register for most other instructions is R7. ADI will allow quick writing of R7 to other registers by adding 0.

An important detail of R-types is their use of register offsets. When registers are likely to be used together, write them close to each other.

Due to the limited control of branch space, use NO-OPS to pad instructions to ensure control flow.

A one-to-one conversion of MIPS or ARM styled ISA would not work for this machine due to the limitations emplaced by its fixed instruction-width. In MIPS, R-types utilized 3 register operands allowing for the registers to have much more versatility than the registers found in this architecture were many instructions have limitations in accessible registers. Rather the philosophy of MIPS was utilized were the tools that were decided as most useful or necessary were encoded as an instructions in the architecture.

5. Individual Component Specification

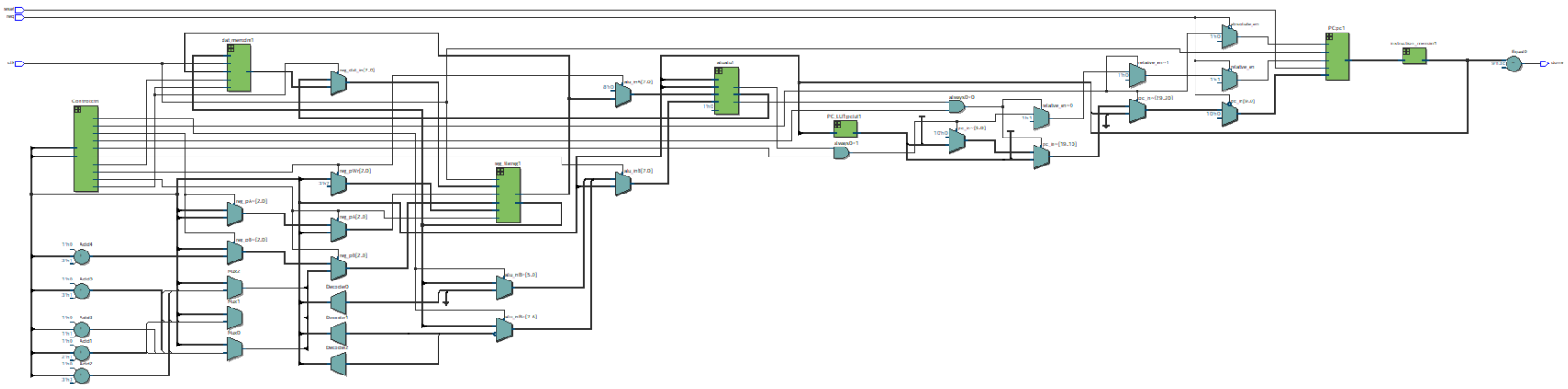
Top Level

Module file name: top_level.sv

Functionality Description

Connects all submodules together and drives the logic.

Schematic



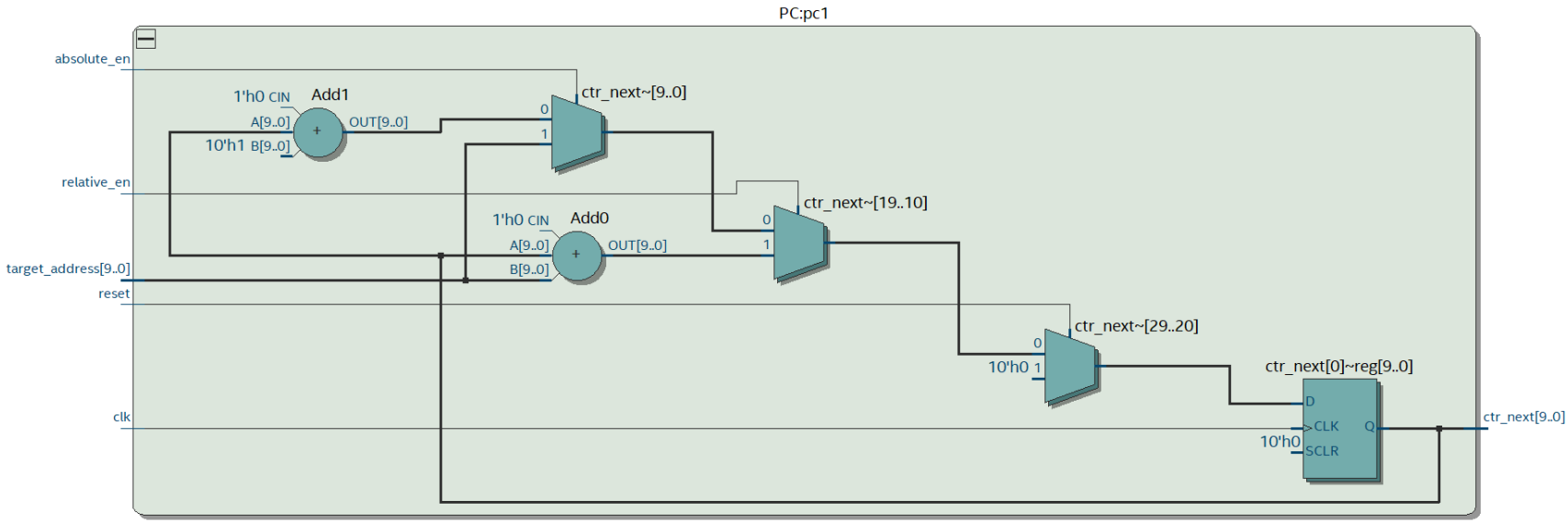
Program Counter

Module file name: PC.sv
Module testbench file name: pc_tb.sv

Functionality Description

Receives two input flags that declares how the program counter will jump, otherwise will increment by 1

Schematic



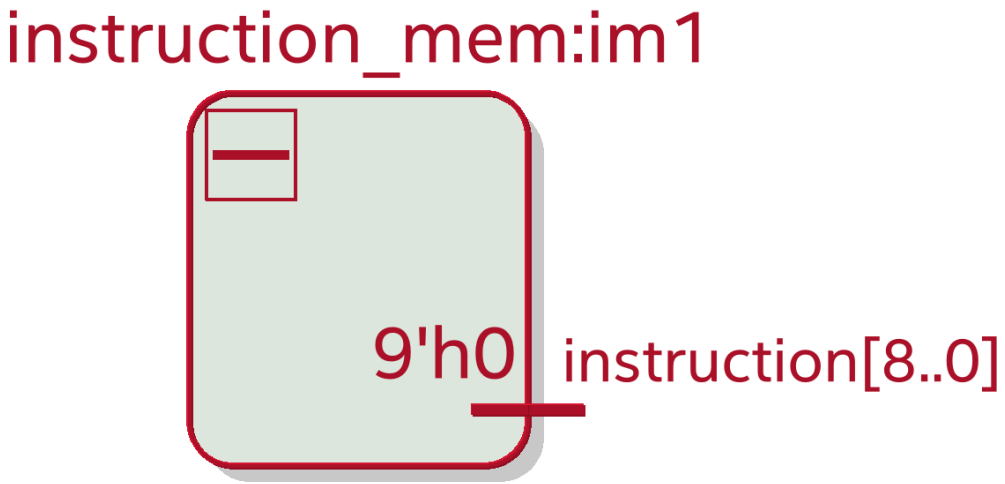
Instruction Memory

Module file name: instruction_mem.sv

Functionality Description

Stores 9-bit wide instructions.

Schematic



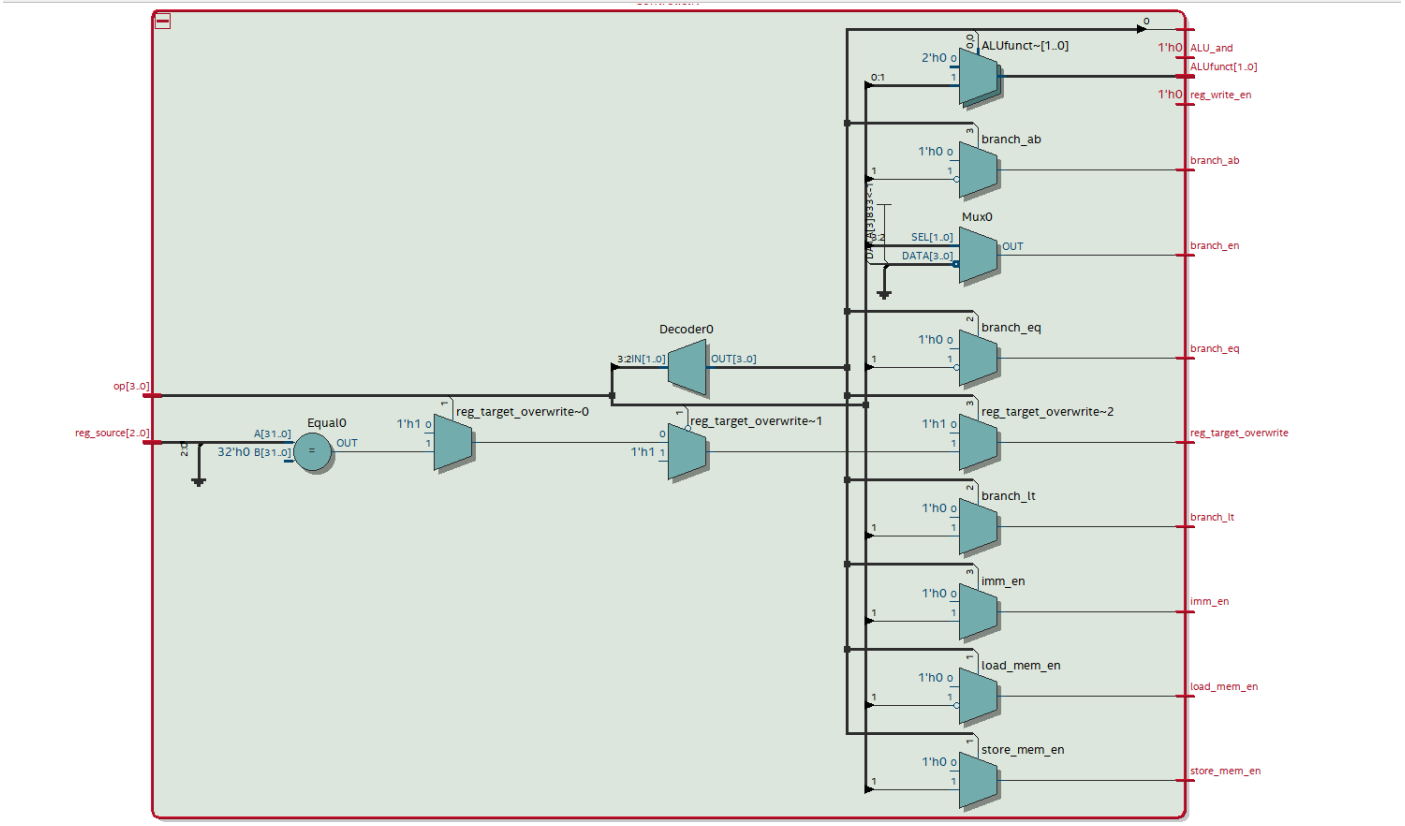
Control Decoder

Module file name: Control.sv

Functionality Description

Decodes instruction to behavior of rest of architecture

Schematic



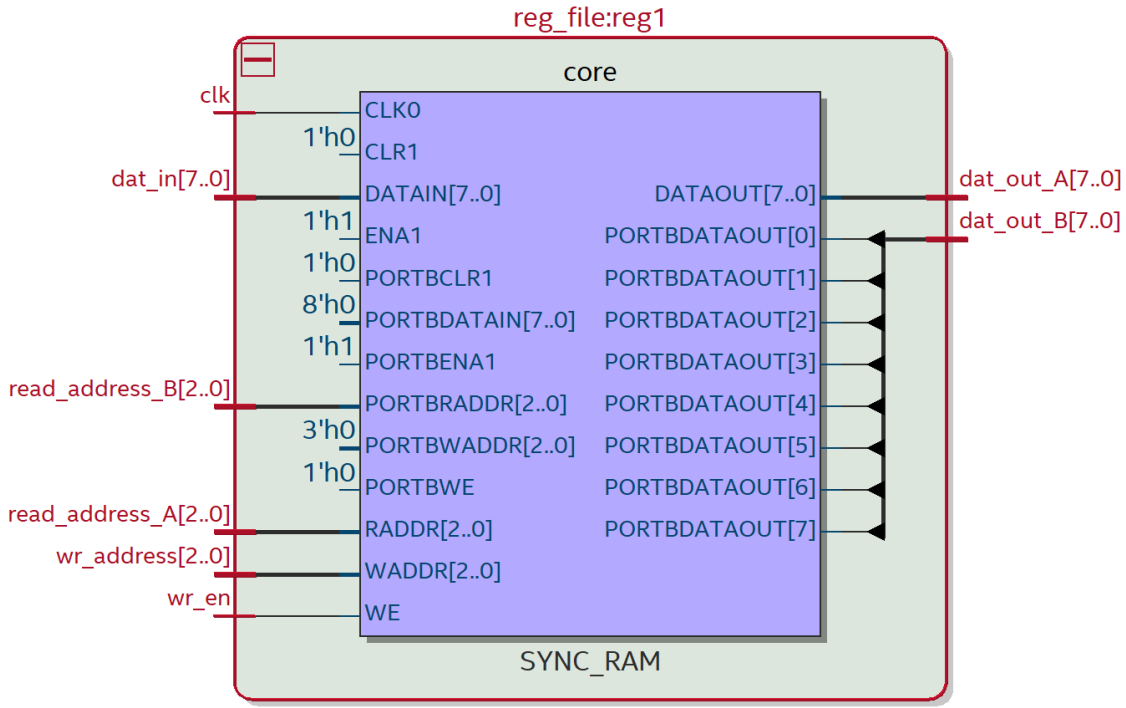
Register File

Module file name: reg_file.sv

Functionality Description

Reads registers or writes data in to register. Has 5 general purpose registers, one semi-accumulator, and

Schematic



ALU (Arithmetic Logic Unit)

Module file name: alu.sv
Module testbench file name: alu_tb.sv

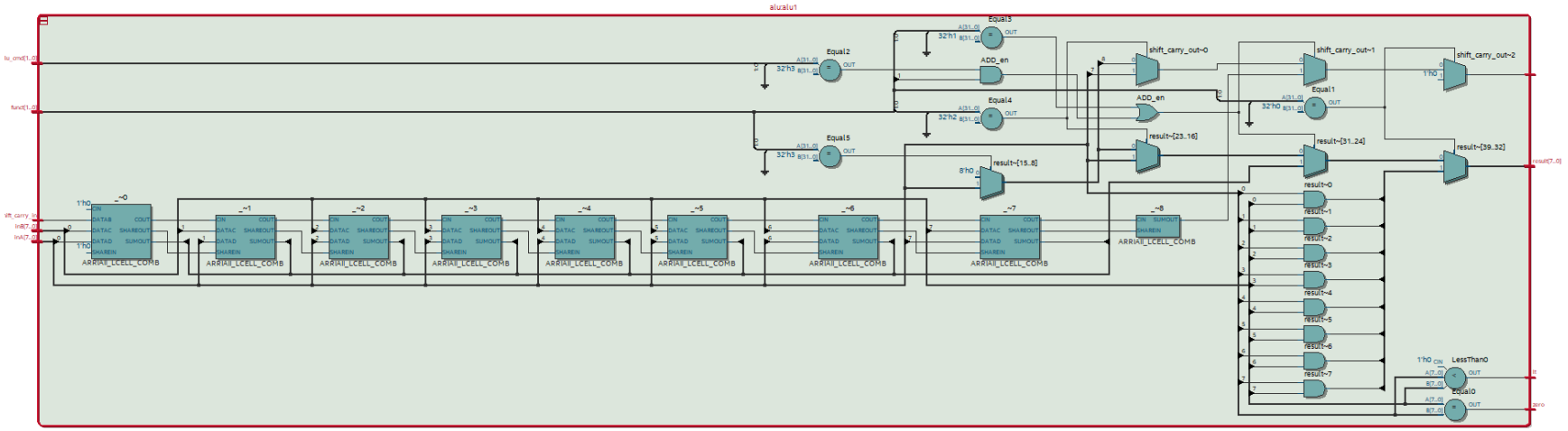
Functionality Description

Implements arithmetic logic necessary for architecture

ALU Operations

Addition between two registers, addition between register and immediate, rolling register left/write by immediate, bitwise AND

Schematic



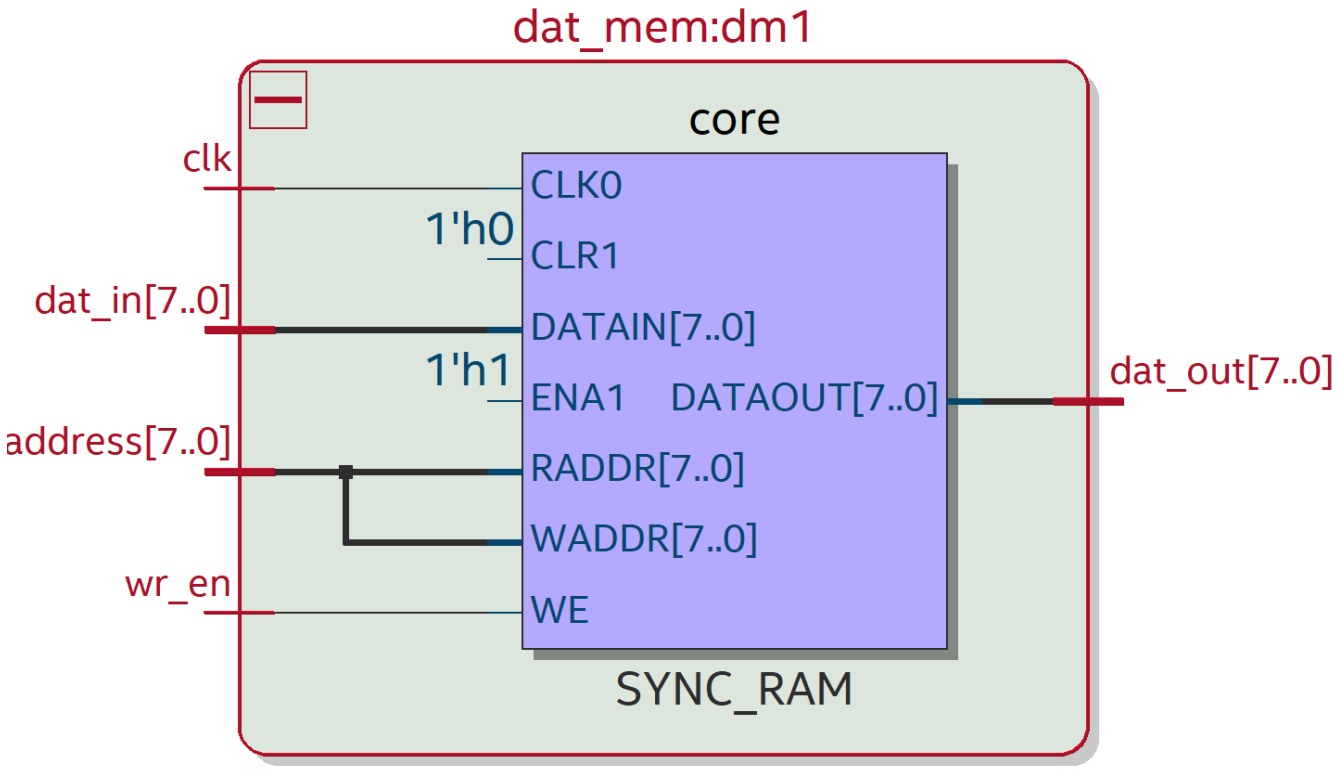
Data Memory

Module file name: dat_mem.sv

Functionality Description

Contains 256 8-bit wide words that are accessible by input address.

Schematic



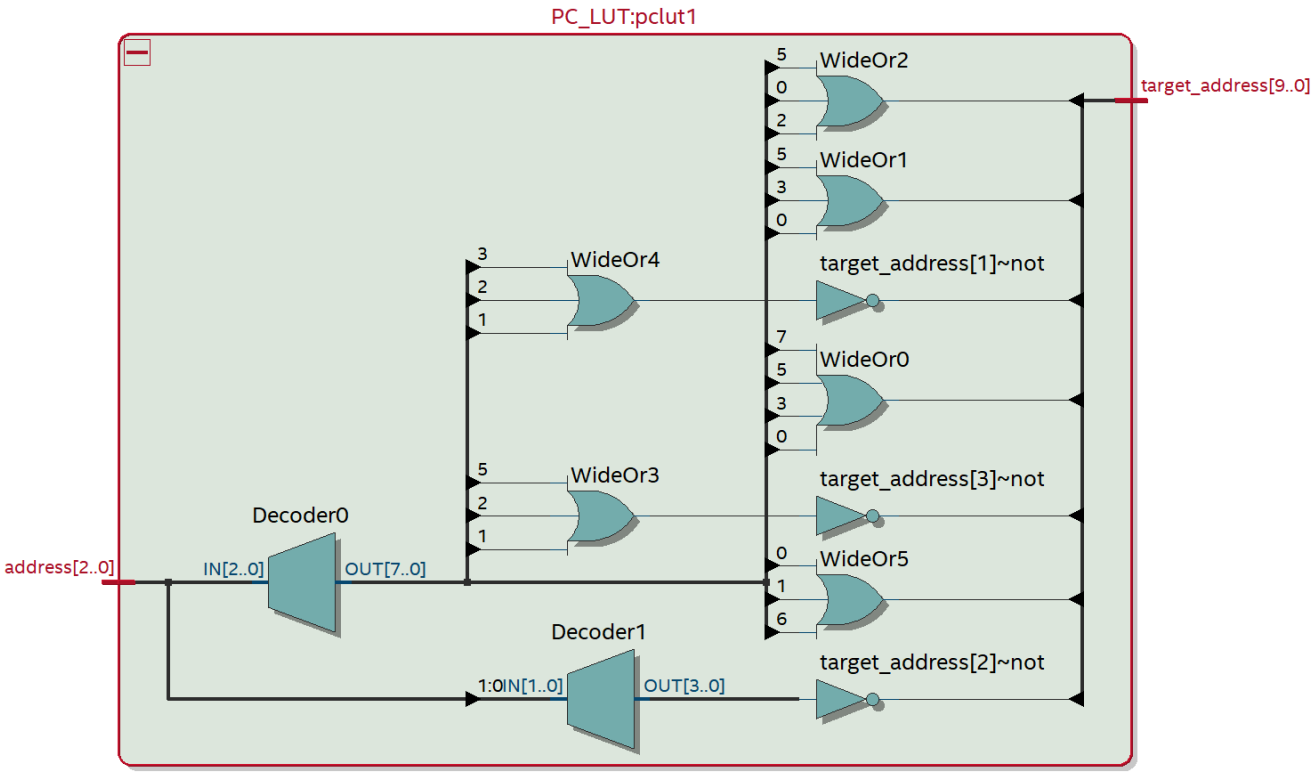
Lookup Tables

Module file name: PC_LUT.sv

Functionality Description

Expands 3-bit wide value into curated relative branch values to allow for control flow.

Schematic



6. Program Implementation

Assembler

Instructions in assembly programs are written in a single line.
Registers are marked with ‘r’ and immediate values are marked with ‘#’.
Included in the submission is a vscode extension that highlights keyword and values.
Freaky Gojo Assembly Scripts have the file extension of ‘.fg’ and the default output binary extension is ‘.fgo’.
Labels are only legal when -r flag is set

The assembler was written in C and requires `g++` in its default make configuration.
Compile the executable with `make windows` or `make linux.`
Usage: `./fga [input] [OPTIONS]`

- Options:
  - `-h` Displays help
  - `-o` Redirects output binary
  - `-a` Appends output binaries with original assembly instruction
  - `-r` Semi-compiler mode that will organize instructions to work with labels

Assembly Language Analysis

The language is rather verbose, requiring frequent movement between registers. The ADI instruction handles many needed operations needed for functionality, from moving registers, incrementing, decrementing, creating bit masks. The ADI instruction is the most used instruction in all non-trivial instructions.

The verbosity of the language is to a fault, forgetting to move a value out of the accumulator register is a common thing to troubleshoot.

Program 1 Pseudocode

```
Load data into registers

If both high and low byte are 0:
    Put into output memory position 0

// Gets absolute value
If MSB == 1
    High_low XOR 0xffff
    Add 1 and propagate it into higher bit if overflow

Shift high_low left until MSB == 1
Exponent = 23 - shift_amount

Shift high_low to remove implicit 1

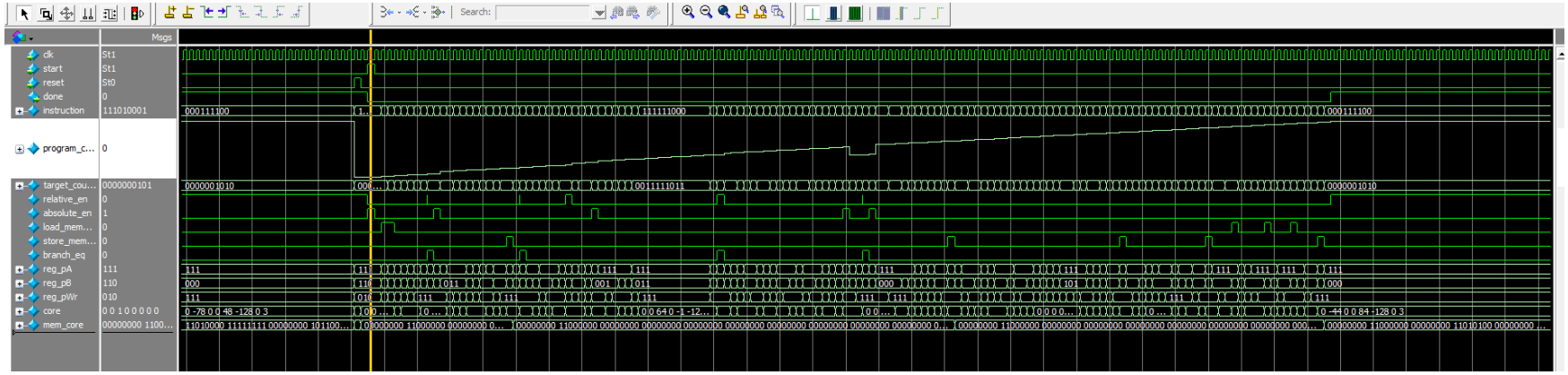
Mantissa = highest 10 bits of high_low

Concatenate bit values and store them into correct memory array
```

Program 1 Assembly Code

See embedded file for specific implementation

Program 1 Output




```
#
# data_mem.core0, 1 = 01010101  00000000           1944855000
# data_mem.core0, 1 = 01001101  00000000           1944895000
# data_mem.core0, 1 = 01001101  01001101           1946655000
# what's feeding the case 0001010001011100
# IN=0x145c,  DUT=0x4d17, REF=0x4d17, MATH=0x4d17
# Scores so far: vs REF=98, vs MATH=98, tests=98
# This test case 0100000001011011
#
# data_mem.core0, 1 = 01001101  00000000           1964905000
# data_mem.core0, 1 = 01010100  00000000           1964945000
# data_mem.core0, 1 = 01010100  01010100           1966265000
# what's feeding the case 0100000001011011
# IN=0x405b,  DUT=0x5405, REF=0x5405, MATH=0x5405
# Scores so far: vs REF=99, vs MATH=99, tests=99
# This test case 0011011110001001
#
# data_mem.core0, 1 = 01010100  00000000           1984955000
# data_mem.core0, 1 = 01010010  00000000           1984995000
# data_mem.core0, 1 = 01010010  01010010           1986535000
# what's feeding the case 0011011110001001
# IN=0x3789,  DUT=0x52f1, REF=0x52f1, MATH=0x52f1
# Scores so far: vs REF=100, vs MATH=100, tests=100
# This test case 0011001001001001
#
# data_mem.core0, 1 = 01010010  00000000           2005005000
# data_mem.core0, 1 = 01010010  01010010           2006585000
# what's feeding the case 0011001001001001
# IN=0x3249,  DUT=0x5249, REF=0x5249, MATH=0x5249
# Scores so far: vs REF=101, vs MATH=101, tests=101
# scores =         101         101 out of         101
# ** Note: $stop    : D:/School Files/College/spring2025/CSE1411/141CPU/revised_05222025 -- use these/Program 1_ testbench + dummy DUT/new_fix2flt_tb.sv(87)
#    Time: 2025075 ns  Iteration: 0  Instance: /new_int2flt_tb
# Break in Module new_int2flt_tb at D:/School Files/College/spring2025/CSE1411/141CPU/revised_05222025 -- use these/Program 1_ testbench + dummy DUT/new_fix2flt_tb.sv lin
e 87
```

Program 2 Pseudocode

```
// Inverse of Program 1

Load data into registers from memory

// Get sign bit
Sign_bit = highest bit of high_low

// Get exponent
Exponent = (high >> 2) && 0x1f

// Handle mantissa
Isolate last two bits of high byte with (high && 0x03)
Lower mantissa is in low byte

Insert implicit leading 1 into high mantissa

shift high_low mantissa by (15 - exponent)

Store data into declared memory output locations
```

Program 3 Pseudocode

```
Extract sign from both float’s high byte
Extract exponents from both float’s high byte
Extract mantissa’s from both floats high_low

If exponent1 < exponent 2:
   Shift mantissa1
If exponent2 < exponent 1:
   Shift mantissa2
// Otherwise the two values have the same exponent and don’t need to be shifted

Add/Subtract mantissas depending on signed bit

Normalize the result by shifting the mantissa left and incrementing exponent

From calculated exponent and mantissa, repack into float
```

Test Benches

The test benches are taken verbatim from the professor's provided files, except mentions of the data_mem modules and its core were standardized to data_mem1 and mem_core, respectively; and the module name of top_level being declared as such. None of the tests were modified or changed.

The test benches work by inserting data values directly into the memory core of the processor, where it then waits for the processor's done flag to read the output data from memory.

The dut's values are then compared with the values derived from the testbenches correct values.

The testbenches perform several tests and the total correct value is incremented and displayed at the end.

7. Reflection

Many mistakes were created in the design scribbles I mocked up that have resulted in technical debt that restricts the ISA from expanding or changing, thus trapping the assembly language into being verbose due to the restrictions. Many possible permutations of the 9bit span are completely unused and mapped to nothing.

For the express purpose of performing the described processes? This processor will be performant in their contexts but may require more non-trivial approaches to perform simple operations. For float operations, the most vital functionality are to add, invert, and shift values in order to construct or deconstruct the float.

Seeing the issues and the amount of DON’T CARES in my ISA, this processor could be more performant on the same tasks with only an 8-bit wide instruction field.

8. Changelog

- Final Submission
  - Added two new instructions XOR and BCO (branch when carry out)
    - Updated Assembler to include these new branches
    - Removed option to go to Link Register, use of function spaces was not helpful for the implementation of these programs
    - Removed Rolling shift right.
  - Updated command overview to match updated restrictions and instructions.
    - Changed all register delimiters from ‘$’ to the updated ‘r’
    - Added XOR and BCO writeups
    - Modified Branch instructions to mention their relationship with labels
    - Removed LSR (then logical shift right) from the command overview table
  - Included Pseudocodes for programs 1, 2, and 3
  - Implemented programs 1 (complete) and program 2 (incomplete)
  - Resynthesized modules and recaptured Schematics
  - Included an analysis of the assembly language
  - Included a reflection of the project as a whole from its shortcomings and capabilities
- Milestone 3.5
  - Added a link register that saves PC + 1 when a JMP command is input into system
  - Added GOL command that sets next PC to saved link register
    - Decreased JMP immediate buffer size from 6 to 5.
- Milestone 3
  - Fixed connections on top_level.sv to properly drive logic of submodules
    - Changed controlling flags to properly implement correct pointers to reg file
  - Added reset input condition to reg_file.sv
    - Initializes all values to zero
  - Added reset input condition to dat_mem.sv
    - Initializes all values to zero
  - Added more flags and options to assembler

        - Assembler in its current version consists of 7 files: assembler.c, assembler.h, encoders.c, encoders.h, errors.h, flags.c, flags.
    - Assembler implements a limited version of labels
        - Implementing this feature required the entire assembler to be restructured
    - Renamed assembler program executable name from “assembler” to “fga”
        - Removes confusion in file hierarchy
- Milestone 2
    - Added milestone 2 expectations
        - Submodule names
        - Submodule descriptions
        - Submodule schematics
    - Created testbenches for PC.sv and alu.sv
    - Changed AND from S-type to R-type
        - AND will now compare two registers rather than hardcoded bitmaps
    - Removed AND_LUT
    - LSR and LSL changed to immediate types and changed behavior from logical shifting to rolling shifters
    - JML changed to JMP
    - Fixed calculation error with JMP sectioning (32 => 64)
    - Branch comparisons will now compare a base register and its register address + 1
        - Old behavior: compares provided register and register 7
    - Done flag will be marked by specified instruction in assembly with “DON” or by inserting word “000111100”
    - No operations will be marked by specified instruction in assembly with “NOP” or by inserting word “111111000”
    - Added Req logic deciding if PC will advance
    - Assembler program has been added
        - Compile by running `make <windows/linux>` , creating an executable binary for those systems.
        - Usage: `./assembler <input> [options]`
            - Options:
                - `-o <output>`
                - `(More to be added)`
    - Added early version of program 1
- Milestone 1
    - Initial version